SIMATS ENGINEERING

ASSESSMENT TOOL 1 – High (Coding Challenge)

COURSE CODE: CSA09 **COURSE NAME: Programming in Java**

COURSE OUTCOME COVERED

CO1: *Apply object-oriented programming principles such as classes, objects, encapsulation, data hiding, inheritance, and polymorphism to design and implement entity manipulations (BL3,BL4)*

QUESTIONS: 10 **Total Marks: 100**

ANNEXURE A **Coding Challenge**

Code of Conduct:

I M.Yeshwanth Kumar (192325047) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:M.Yeshwanth

RUBRICS FOR CALCULATION:

Criteria	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding				
Algorithm				
Code Implementation				
Competitive Performance				
Test Case Handling				

Coding Challenge

1. Library Book Management using Classes & Encapsulation

Design a Book class with private fields such as bookId, title, author, and price. Implement constructors, getters, setters, and methods to issue and return books. Create a menu-driven program to manage multiple books using an array of objects. Also, construct suitable test cases to validate book addition, issue, return, and search operations. BTL: BL3 (Apply)

Code:

```
import java.util.Scanner;

class Book {
    private int bookId;
    private String title;
    private String author;
    private double price;
    private boolean issued;

    public Book(int bookId, String title, String author, double price) {
        this.bookId = bookId;
        this.title = title;
        this.author = author;
        this.price = price;
        this.issued = false;
    }

    public int getBookId() {
        return bookId;
    }

    public void issueBook() {
        if (!issued) {
            issued = true;
            System.out.println("Book Issued Successfully.");
        } else {
            System.out.println("Book is Already Issued.");
        }
    }

    public void returnBook() {
        if (issued) {
            issued = false;
            System.out.println("Book Returned Successfully.");
        } else {
            System.out.println("Book was not Issued.");
        }
    }

    public void display() {
        System.out.println("Book ID : " + bookId);
        System.out.println("Title   : " + title);
```

```
        System.out.println("Author  : " + author);
        System.out.println("Price   : " + price);
        System.out.println("Status  : " + (issued ? "Issued" : "Available"));
    }
}

public class LibraryManagement {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        Book[] books = new Book[100];
        int count = 0, choice;

        do {
            System.out.println("\n1.Add Book");
            System.out.println("2.Display Books");
            System.out.println("3.Search Book");
            System.out.println("4.Issue Book");
            System.out.println("5.Return Book");
            System.out.println("6.Exit");
            System.out.print("Enter Choice: ");
            choice = sc.nextInt();

            switch (choice) {

                case 1:
                    System.out.print("Book ID: ");
                    int id = sc.nextInt();
                    sc.nextLine();

                    System.out.print("Title: ");
                    String title = sc.nextLine();

                    System.out.print("Author: ");
                    String author = sc.nextLine();

                    System.out.print("Price: ");
                    double price = sc.nextDouble();

                    books[count] = new Book(id, title, author, price);
                    count++;
                    System.out.println("Book Added.");
                    break;

                case 2:
                    for (int i = 0; i < count; i++) {
                        books[i].display();
                    }
                    break;

                case 3:
                    System.out.print("Enter Book ID: ");
                    int search = sc.nextInt();
                    boolean found = false;
```

```
                for (int i = 0; i < count; i++) {
                    if (books[i].getBookId() == search) {
                        books[i].display();
                        found = true;
                    }
                }

                if (!found)
                    System.out.println("Book Not Found.");
                break;

            case 4:
                System.out.print("Enter Book ID: ");
                int issue = sc.nextInt();

                for (int i = 0; i < count; i++) {
                    if (books[i].getBookId() == issue) {
                        books[i].issueBook();
                    }
                }
                break;

            case 5:
                System.out.print("Enter Book ID: ");
                int ret = sc.nextInt();

                for (int i = 0; i < count; i++) {
                    if (books[i].getBookId() == ret) {
                        books[i].returnBook();
                    }
                }
                break;

            case 6:
                System.out.println("Thank You!");
                break;

            default:
                System.out.println("Invalid Choice");
        }

    } while (choice != 6);

    sc.close();
  }
}
```

Output:

```
PS C:\Users\WINDOWS 11\OneDrive\Documents\java programs> javac LibraryManagement.java
>> java LibraryManagement

1.Add Book
2.Display Books
3.Search Book
4.Issue Book
5.Return Book
6.Exit
Enter Choice: 1
Book ID: 1234
Title: the brave man
Author: M.yeshwanth Kumar
Price: 1000
Book Added.

1.Add Book
2.Display Books
3.Search Book
4.Issue Book
5.Return Book
6.Exit
Enter Choice: 4
Enter Book ID: 1234
Book Issued Successfully.

1.Add Book
2.Display Books
3.Search Book
4.Issue Book
5.Return Book
6.Exit
Enter Choice: 2
Book ID : 1234
Title   : the brave man
```

2. Employee Payroll System using Inheritance

Design a payroll system with a base class Employee and derived classes PermanentEmployee and ContractEmployee. Override a method calculateSalary() in each subclass to compute salary differently. Display employee details and salary using runtime polymorphism. Also, construct suitable test cases to validate salary calculation for different employee types. BTL: BL3 (Apply)

Code:

```java
import java.util.Scanner;

class Employee {
    int empId;
    String name;

    Employee(int empId, String name) {
        this.empId = empId;
        this.name = name;
    }

    void calculateSalary() {
        System.out.println("Salary Calculation");
    }

    void display() {
        System.out.println("Employee ID : " + empId);
        System.out.println("Employee Name : " + name);
```

```
    }
}

class PermanentEmployee extends Employee {
    double basicSalary, hra, da;

    PermanentEmployee(int empId, String name, double basicSalary, double hra,
double da) {
        super(empId, name);
        this.basicSalary = basicSalary;
        this.hra = hra;
        this.da = da;
    }

    @Override
    void calculateSalary() {
        double salary = basicSalary + hra + da;
        display();
        System.out.println("Employee Type : Permanent");
        System.out.println("Salary : " + salary);
    }
}

class ContractEmployee extends Employee {
    int hoursWorked;
    double ratePerHour;

    ContractEmployee(int empId, String name, int hoursWorked, double
ratePerHour) {
        super(empId, name);
        this.hoursWorked = hoursWorked;
        this.ratePerHour = ratePerHour;
    }

    @Override
    void calculateSalary() {
        double salary = hoursWorked * ratePerHour;
        display();
        System.out.println("Employee Type : Contract");
        System.out.println("Salary : " + salary);
    }
}

public class EmployeePayroll {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.println("1. Permanent Employee");
        System.out.println("2. Contract Employee");
        System.out.print("Enter Choice: ");
        int choice = sc.nextInt();

        Employee emp;
```

```
        if (choice == 1) {

            System.out.print("Enter Employee ID: ");
            int id = sc.nextInt();
            sc.nextLine();

            System.out.print("Enter Name: ");
            String name = sc.nextLine();

            System.out.print("Enter Basic Salary: ");
            double basic = sc.nextDouble();

            System.out.print("Enter HRA: ");
            double hra = sc.nextDouble();

            System.out.print("Enter DA: ");
            double da = sc.nextDouble();

            emp = new PermanentEmployee(id, name, basic, hra, da);

        } else {

            System.out.print("Enter Employee ID: ");
            int id = sc.nextInt();
            sc.nextLine();

            System.out.print("Enter Name: ");
            String name = sc.nextLine();

            System.out.print("Enter Hours Worked: ");
            int hours = sc.nextInt();

            System.out.print("Enter Rate Per Hour: ");
            double rate = sc.nextDouble();

            emp = new ContractEmployee(id, name, hours, rate);

        }

        System.out.println("\nEmployee Details");
        emp.calculateSalary();

        sc.close();
    }
}
```

Output :

```
PS C:\Users\WINDOWS 11\OneDrive\Documents\java programs>  & 'C:\Program Files\Java\jdk-17.0.19\bin\java.exe' '-agentlib:jdwp=transport=dt_socket,server=n,suspend=y,address=localhost:62859' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\WINDOWS 11\AppData\Roaming\Code\User\workspaceStorage\1f0c1a872a15026a62055ee1093c288c\redhat.java\jdt_ws\java programs_33609ebe\bin' 'EmployeePayroll'
1. Permanent Employee
2. Contract Employee
Enter Choice: 1
Enter Employee ID: 1234
Enter Name: M.Yeshwanth
Enter Basic Salary: 25000
Enter HRA: 5000
Enter DA: 2000

Employee Details
Employee ID : 1234
Employee Name : M.Yeshwanth
Employee Type : Permanent
Salary : 32000.0
PS C:\Users\WINDOWS 11\OneDrive\Documents\java programs>
```

3. Vehicle Rental System using Runtime Polymorphism

Develop a vehicle rental system with a superclass Vehicle and subclasses Car, Bike, and Bus. Override the method calculateRent(int days) in each subclass. Store vehicles in an array of Vehicle references and generate rental bills dynamically. Also, construct suitable test cases to validate rent calculation for different vehicle categories. BTL: BL4 (Analyze)

Code :

```java
import java.util.Scanner;

class Vehicle {
    int vehicleId;
    String vehicleName;

    Vehicle(int vehicleId, String vehicleName) {
        this.vehicleId = vehicleId;
        this.vehicleName = vehicleName;
    }

    void calculateRent(int days) {
        System.out.println("Rent Calculation");
    }

    void display() {
        System.out.println("Vehicle ID   : " + vehicleId);
        System.out.println("Vehicle Name : " + vehicleName);
    }
}

class Car extends Vehicle {

    Car(int vehicleId, String vehicleName) {
        super(vehicleId, vehicleName);
    }

    @Override
    void calculateRent(int days) {
        display();
```

```
        System.out.println("Vehicle Type : Car");
        System.out.println("Days         : " + days);
        System.out.println("Total Rent   : " + (days * 1000));
    }
}

class Bike extends Vehicle {

    Bike(int vehicleId, String vehicleName) {
        super(vehicleId, vehicleName);
    }

    @Override
    void calculateRent(int days) {
        display();
        System.out.println("Vehicle Type : Bike");
        System.out.println("Days         : " + days);
        System.out.println("Total Rent   : " + (days * 500));
    }
}

class Bus extends Vehicle {

    Bus(int vehicleId, String vehicleName) {
        super(vehicleId, vehicleName);
    }

    @Override
    void calculateRent(int days) {
        display();
        System.out.println("Vehicle Type : Bus");
        System.out.println("Days         : " + days);
        System.out.println("Total Rent   : " + (days * 2000));
    }
}

public class VehicleRental {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        Vehicle[] vehicles = new Vehicle[10];

        int count = 0;
        int choice;

        do {

            System.out.println("\n===== Vehicle Rental System =====");
            System.out.println("1. Add Car");
            System.out.println("2. Add Bike");
            System.out.println("3. Add Bus");
            System.out.println("4. Generate Rental Bill");
            System.out.println("5. Exit");
```

```
            System.out.print("Enter Choice: ");
            choice = sc.nextInt();

            switch (choice) {

                case 1:
                    System.out.print("Enter Vehicle ID: ");
                    int cid = sc.nextInt();
                    sc.nextLine();

                    System.out.print("Enter Car Name: ");
                    String cname = sc.nextLine();

                    vehicles[count] = new Car(cid, cname);
                    count++;
                    System.out.println("Car Added Successfully.");
                    break;

                case 2:
                    System.out.print("Enter Vehicle ID: ");
                    int bid = sc.nextInt();
                    sc.nextLine();

                    System.out.print("Enter Bike Name: ");
                    String bname = sc.nextLine();

                    vehicles[count] = new Bike(bid, bname);
                    count++;
                    System.out.println("Bike Added Successfully.");
                    break;

                case 3:
                    System.out.print("Enter Vehicle ID: ");
                    int busid = sc.nextInt();
                    sc.nextLine();

                    System.out.print("Enter Bus Name: ");
                    String busname = sc.nextLine();

                    vehicles[count] = new Bus(busid, busname);
                    count++;
                    System.out.println("Bus Added Successfully.");
                    break;

                case 4:
                    System.out.print("Enter Vehicle ID: ");
                    int id = sc.nextInt();

                    System.out.print("Enter Number of Days: ");
                    int days = sc.nextInt();

                    boolean found = false;

                    for (int i = 0; i < count; i++) {
                        if (vehicles[i].vehicleId == id) {
```

```
                        System.out.println("\n----- Rental Bill -----");
                        vehicles[i].calculateRent(days);
                        found = true;
                        break;
                    }
                }

                if (!found) {
                    System.out.println("Vehicle Not Found.");
                }

                break;

            case 5:
                System.out.println("Thank You!");
                break;

            default:
                System.out.println("Invalid Choice.");

        }

    } while (choice != 5);

    sc.close();
  }
}
```

Output:

```
PS C:\Users\WINDOWS 11\OneDrive\Documents\java programs>  & 'C:\Program Files\Java\jdk-17.0.19\bin\ja
va.exe' '-agentlib:jdwp=transport=dt_socket,server=n,suspend=y,address=localhost:64197' '-XX:+ShowCod
eDetailsInExceptionMessages' '-cp' 'C:\Users\WINDOWS 11\AppData\Roaming\Code\User\workspaceStorage\1f
0c1a872a15026a62055ee1093c288c\redhat.java\jdt_ws\java programs_33609ebe\bin' 'VehicleRental'

===== Vehicle Rental System =====
1. Add Car
2. Add Bike
3. Add Bus
4. Generate Rental Bill
5. Exit
Enter Choice: 1
Enter Vehicle ID: 1234
Enter Car Name: Baleno
Car Added Successfully.

===== Vehicle Rental System =====
1. Add Car
2. Add Bike
3. Add Bus
4. Generate Rental Bill
5. Exit
Enter Choice: 4
Enter Vehicle ID: 1234
Enter Number of Days: 2

----- Rental Bill -----
Vehicle ID   : 1234
Vehicle Name : Baleno
Vehicle Type : Car
Days         : 2
Total Rent   : 2000
```

4. Bank Account System using Data Hiding

Design a BankAccount class with private fields accountNumber, holderName, and balance. Implement methods for deposit, withdrawal, and balance inquiry with proper validation. Prevent direct modification of balance from outside the class. Also, construct suitable test cases to validate valid and invalid transactions. BTL: BL3 (Apply)

Code:

```java
import java.util.Scanner;

class BankAccount {

    private int accountNumber;
    private String holderName;
    private double balance;

    // Constructor
    BankAccount(int accountNumber, String holderName, double balance) {
        this.accountNumber = accountNumber;
        this.holderName = holderName;
        this.balance = balance;
    }

    // Deposit Method
    void deposit(double amount) {
        if (amount > 0) {
            balance = balance + amount;
```

```
            System.out.println("Amount Deposited Successfully.");
        } else {
            System.out.println("Invalid Deposit Amount.");
        }
    }

    // Withdraw Method
    void withdraw(double amount) {
        if (amount <= balance && amount > 0) {
            balance = balance - amount;
            System.out.println("Amount Withdrawn Successfully.");
        } else {
            System.out.println("Insufficient Balance or Invalid Amount.");
        }
    }

    // Balance Inquiry
    void display() {
        System.out.println("\nAccount Number : " + accountNumber);
        System.out.println("Holder Name    : " + holderName);
        System.out.println("Balance        : " + balance);
    }
}

public class BankAccountSystem {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter Account Number: ");
        int accNo = sc.nextInt();
        sc.nextLine();

        System.out.print("Enter Holder Name: ");
        String name = sc.nextLine();

        System.out.print("Enter Initial Balance: ");
        double balance = sc.nextDouble();

        BankAccount account = new BankAccount(accNo, name, balance);

        int choice;

        do {

            System.out.println("\n===== Bank Account Menu =====");
            System.out.println("1. Deposit");
            System.out.println("2. Withdraw");
            System.out.println("3. Balance Inquiry");
            System.out.println("4. Exit");
            System.out.print("Enter Choice: ");

            choice = sc.nextInt();
```

```
            switch (choice) {

                case 1:
                    System.out.print("Enter Deposit Amount: ");
                    double deposit = sc.nextDouble();
                    account.deposit(deposit);
                    break;

                case 2:
                    System.out.print("Enter Withdrawal Amount: ");
                    double withdraw = sc.nextDouble();
                    account.withdraw(withdraw);
                    break;

                case 3:
                    account.display();
                    break;

                case 4:
                    System.out.println("Thank You!");
                    break;

                default:
                    System.out.println("Invalid Choice.");
            }

        } while (choice != 4);

        sc.close();
    }
}
```

Output:

```
PROBLEMS 18   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

PS C:\Users\WINDOWS 11\OneDrive\Documents\java programs>  & 'C:\Program Files\Java\jdk-17.0.19\bin\java.exe' '-agentlib:jdwp=transport=dt_socket,server=n,suspend=y,address=localhost:64143' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\WINDOWS 11\AppData\Roaming\Code\User\workspaceStorage\1f0c1a872a15026a62055ee1093c288c\redhat.java\jdt_ws\java programs_33609ebe\bin' 'BankAccountSystem'
Enter Account Number: 98480312
Enter Holder Name: yeshwanth
Enter Initial Balance: 100000

===== Bank Account Menu =====
1. Deposit
2. Withdraw
3. Balance Inquiry
4. Exit
Enter Choice: 2
Enter Withdrawal Amount: 5000
Amount Withdrawn Successfully.

===== Bank Account Menu =====
1. Deposit
2. Withdraw
3. Balance Inquiry
4. Exit
Enter Choice: 3

Account Number : 98480312
Holder Name    : yeshwanth
Balance        : 95000.0
```

5. University Admission System using Method Overloading

Create an Admission class that overloads the method calculateFee() for:

- Undergraduate students,
- Postgraduate students,
- Scholarship students.

Display the fee structure based on user input and compare the overloaded method executions. Also, construct suitable test cases to validate each overloaded version. BTL: BL3 (Apply)

Code :

```java
import java.util.Scanner;

class Admission {

    // Undergraduate Fee
    void calculateFee(double fee) {
        System.out.println("\nStudent Type : Undergraduate");
        System.out.println("Course Fee   : " + fee);
    }

    // Postgraduate Fee
    void calculateFee(double fee, int years) {
        System.out.println("\nStudent Type : Postgraduate");
        System.out.println("Course Fee   : " + (fee * years));
    }

    // Scholarship Student Fee
    void calculateFee(double fee, double scholarship) {
        double finalFee = fee - scholarship;
```

```
        System.out.println("\nStudent Type : Scholarship");
        System.out.println("Original Fee : " + fee);
        System.out.println("Scholarship  : " + scholarship);
        System.out.println("Final Fee    : " + finalFee);
    }
}

public class AdmissionSystem {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        Admission ad = new Admission();

        int choice;

        do {

            System.out.println("\n===== University Admission System =====");
            System.out.println("1. Undergraduate Student");
            System.out.println("2. Postgraduate Student");
            System.out.println("3. Scholarship Student");
            System.out.println("4. Exit");
            System.out.print("Enter Choice: ");

            choice = sc.nextInt();

            switch (choice) {

                case 1:
                    System.out.print("Enter Undergraduate Fee: ");
                    double ugFee = sc.nextDouble();
                    ad.calculateFee(ugFee);
                    break;

                case 2:
                    System.out.print("Enter Fee Per Year: ");
                    double pgFee = sc.nextDouble();

                    System.out.print("Enter Number of Years: ");
                    int years = sc.nextInt();

                    ad.calculateFee(pgFee, years);
                    break;

                case 3:
                    System.out.print("Enter Original Fee: ");
                    double fee = sc.nextDouble();

                    System.out.print("Enter Scholarship Amount: ");
                    double scholarship = sc.nextDouble();

                    ad.calculateFee(fee, scholarship);
                    break;
```

```
                case 4:
                    System.out.println("Thank You!");
                    break;

                default:
                    System.out.println("Invalid Choice.");
            }

        } while (choice != 4);

        sc.close();
    }
}
```

Output:

```
PS C:\Users\WINDOWS 11\OneDrive\Documents\java programs>  & 'C:\Program Files\Java\jdk-17.0.19\bin\java.exe' '-agentlib:jdwp=transport=dt_socket,server=n,suspend=y,address=localhost:58401' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\WINDOWS 11\AppData\Roaming\Code\User\workspaceStorage\1f0c1a872a15026a62055ee1093c288c\redhat.java\jdt_ws\java programs_33609ebe\bin' 'AdmissionSystem'

===== University Admission System =====
1. Undergraduate Student
2. Postgraduate Student
3. Scholarship Student
4. Exit
Enter Choice: 1
Enter Undergraduate Fee: 50000

Student Type : Undergraduate
Course Fee   : 50000.0

===== University Admission System =====
1. Undergraduate Student
2. Postgraduate Student
3. Scholarship Student
4. Exit
```

6. Shape Area Calculator using Abstract Class & Polymorphism

Design an abstract class Shape with an abstract method area(). Implement subclasses Circle, Rectangle, and Triangle. Use an array of Shape references to compute and display areas dynamically. Also, construct suitable test cases to validate area calculations and polymorphic behavior. BTL: BL4 (Analyze)

Code :

```
import java.util.Scanner;

abstract class Shape {

    abstract void area();
}

// Circle Class
class Circle extends Shape {
    double radius;

    Circle(double radius) {
        this.radius = radius;
```

```
    }

    void area() {
        double a = 3.14 * radius * radius;
        System.out.println("Circle Area = " + a);
    }
}

// Rectangle Class
class Rectangle extends Shape {
    double length, width;

    Rectangle(double length, double width) {
        this.length = length;
        this.width = width;
    }

    void area() {
        double a = length * width;
        System.out.println("Rectangle Area = " + a);
    }
}

// Triangle Class
class Triangle extends Shape {
    double base, height;

    Triangle(double base, double height) {
        this.base = base;
        this.height = height;
    }

    void area() {
        double a = 0.5 * base * height;
        System.out.println("Triangle Area = " + a);
    }
}

public class ShapeAreaCalculator {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        Shape[] shapes = new Shape[10];

        int count = 0;
        int choice;

        do {

            System.out.println("\n===== Shape Area Calculator =====");
            System.out.println("1. Circle");
            System.out.println("2. Rectangle");
            System.out.println("3. Triangle");
```

```
            System.out.println("4. Display Areas");
            System.out.println("5. Exit");
            System.out.print("Enter Choice: ");

            choice = sc.nextInt();

            switch (choice) {

                case 1:
                    System.out.print("Enter Radius: ");
                    double r = sc.nextDouble();

                    shapes[count] = new Circle(r);
                    count++;
                    System.out.println("Circle Added.");
                    break;

                case 2:
                    System.out.print("Enter Length: ");
                    double l = sc.nextDouble();

                    System.out.print("Enter Width: ");
                    double w = sc.nextDouble();

                    shapes[count] = new Rectangle(l, w);
                    count++;
                    System.out.println("Rectangle Added.");
                    break;

                case 3:
                    System.out.print("Enter Base: ");
                    double b = sc.nextDouble();

                    System.out.print("Enter Height: ");
                    double h = sc.nextDouble();

                    shapes[count] = new Triangle(b, h);
                    count++;
                    System.out.println("Triangle Added.");
                    break;

                case 4:
                    System.out.println("\nAreas of Shapes");
                    for (int i = 0; i < count; i++) {
                        shapes[i].area();
                    }
                    break;

                case 5:
                    System.out.println("Thank You!");
                    break;

                default:
                    System.out.println("Invalid Choice.");
```

```
            }

        } while (choice != 5);

        sc.close();
    }
}
```

Output:

```
PS C:\Users\WINDOWS 11\OneDrive\Documents\java programs>  & 'C:\Program Files\Java\jdk-17.0.19\bin\java.exe' '-agentlib:jdwp=transport=dt_socket,server=n,suspend=y,address=localhost:63548' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\WINDOWS 11\AppData\Roaming\Code\User\workspaceStorage\1f0c1a872a15026a62055ee1093c288c\redhat.java\jdt_ws\java programs_33609ebe\bin' 'ShapeAreaCalculator'

===== Shape Area Calculator =====
1. Circle
2. Rectangle
3. Triangle
4. Display Areas
5. Exit
Enter Choice: 1
Enter Radius: 5
Circle Added.

===== Shape Area Calculator =====
1. Circle
2. Rectangle
3. Triangle
4. Display Areas
5. Exit
Enter Choice: 4

Areas of Shapes
Circle Area = 78.5
```

7. Hospital Management System using Interfaces

Develop a hospital management system with an interface MedicalRecord containing methods addRecord() and displayRecord(). Implement the interface in classes Patient and Doctor. Demonstrate interface-based abstraction and object interaction. Also, construct suitable test cases to validate record creation and display. BTL: BL3 (Apply)

Code:

```
import java.util.Scanner;

// Interface
interface MedicalRecord {
    void addRecord();
    void displayRecord();
}

// Patient Class
class Patient implements MedicalRecord {
    int patientId;
    String patientName;
    String disease;

    Scanner sc = new Scanner(System.in);
```

```
    public void addRecord() {
        System.out.print("Enter Patient ID: ");
        patientId = sc.nextInt();
        sc.nextLine();

        System.out.print("Enter Patient Name: ");
        patientName = sc.nextLine();

        System.out.print("Enter Disease: ");
        disease = sc.nextLine();
    }

    public void displayRecord() {
        System.out.println("\n----- Patient Record -----");
        System.out.println("Patient ID   : " + patientId);
        System.out.println("Patient Name : " + patientName);
        System.out.println("Disease      : " + disease);
    }
}

// Doctor Class
class Doctor implements MedicalRecord {
    int doctorId;
    String doctorName;
    String specialization;

    Scanner sc = new Scanner(System.in);

    public void addRecord() {
        System.out.print("Enter Doctor ID: ");
        doctorId = sc.nextInt();
        sc.nextLine();

        System.out.print("Enter Doctor Name: ");
        doctorName = sc.nextLine();

        System.out.print("Enter Specialization: ");
        specialization = sc.nextLine();
    }

    public void displayRecord() {
        System.out.println("\n----- Doctor Record -----");
        System.out.println("Doctor ID       : " + doctorId);
        System.out.println("Doctor Name     : " + doctorName);
        System.out.println("Specialization  : " + specialization);
    }
}

// Main Class
public class HospitalManagement {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
```

```
        MedicalRecord obj;
        int choice;

        do {

            System.out.println("\n===== Hospital Management System =====");
            System.out.println("1. Patient Record");
            System.out.println("2. Doctor Record");
            System.out.println("3. Exit");
            System.out.print("Enter Choice: ");

            choice = sc.nextInt();

            switch (choice) {

                case 1:
                    obj = new Patient();
                    obj.addRecord();
                    obj.displayRecord();
                    break;

                case 2:
                    obj = new Doctor();
                    obj.addRecord();
                    obj.displayRecord();
                    break;

                case 3:
                    System.out.println("Thank You!");
                    break;

                default:
                    System.out.println("Invalid Choice.");
            }

        } while (choice != 3);

        sc.close();
    }
}
```

Output:

```
PS C:\Users\WINDOWS 11\OneDrive\Documents\java programs>  & 'C:\Program Files\Java\jdk-17.0.19\bin\java.exe' '-agentlib:jdwp=transport=dt_socket,server=n,suspend=y,address=localhost:54564' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\WINDOWS 11\AppData\Roaming\Code\User\workspaceStorage\1f0c1a872a15026a62055ee1093c288c\redhat.java\jdt_ws\java programs_33609ebe\bin' 'HospitalManagement'

===== Hospital Management System =====
1. Patient Record
2. Doctor Record
3. Exit
Enter Choice: 1
Enter Patient ID: 12334
Enter Patient Name: yeshwanth
Enter Disease: fever

----- Patient Record -----
Patient ID   : 12334
Patient Name : yeshwanth
Disease      : fever

===== Hospital Management System =====
1. Patient Record
2. Doctor Record
3. Exit
Enter Choice: 2
Enter Doctor ID: 234
Enter Doctor Name: yeshu
Enter Specialization: mba

----- Doctor Record -----
Doctor ID       : 234
Doctor Name     : yeshu
Specialization  : mba
```

8. Online Shopping Order System using Packages

Create two packages:

- shopping.product
- shopping.order

Implement classes Product and Order in separate packages. Import the packages into a main application that places orders and calculates the total amount. Also, construct suitable test cases to validate package usage, order placement, and bill generation. BTL: BL3 (Apply)

Code :

```java
import java.util.Scanner;

class Product {

    int id;
    String name;
    double price;

    Product(int id, String name, double price) {
        this.id = id;
        this.name = name;
        this.price = price;
    }

    void display() {
        System.out.println("Product ID   : " + id);
        System.out.println("Product Name : " + name);
```

```
        System.out.println("Price        : " + price);
    }
}

class Order {

    void placeOrder(Product p, int quantity) {
        double total = p.price * quantity;

        System.out.println("\n------ Order Bill ------");
        p.display();
        System.out.println("Quantity     : " + quantity);
        System.out.println("Total Amount : " + total);
    }
}

public class OnlineShoppingSystem {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter Product ID: ");
        int id = sc.nextInt();
        sc.nextLine();

        System.out.print("Enter Product Name: ");
        String name = sc.nextLine();

        System.out.print("Enter Product Price: ");
        double price = sc.nextDouble();

        System.out.print("Enter Quantity: ");
        int quantity = sc.nextInt();

        Product p = new Product(id, name, price);
        Order o = new Order();

        o.placeOrder(p, quantity);

        sc.close();
    }
}
```

Output:

```
PS C:\Users\WINDOWS 11\OneDrive\Documents\java programs>  & 'C:\Program Files\Java\jdk-17.0.19\bin\java.exe' '-agentlib:jdwp=transport=dt_socket,server=n,suspend=y,address=localhost:54008' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\WINDOWS 11\AppData\Roaming\Code\User\workspaceStorage\1f0c1a872a15026a62055ee1093c288c\redhat.java\jdt_ws\java programs_33609ebe\bin' 'OnlineShoppingSystem'

Enter Product ID: 123
Enter Product Name: shoes
Enter Product Price: 2000
Enter Quantity: 1

------ Order Bill ------
Product ID   : 123
Product Name : shoes
Price        : 2000.0
Quantity     : 1
Total Amount : 2000.0
PS C:\Users\WINDOWS 11\OneDrive\Documents\java programs>
```

9. Student Result Analyzer using Arrays of Objects

Design a Student class containing roll number, name, and marks in three subjects. Store multiple student objects in an array and implement methods to calculate total, average, grade, and class topper. Also, construct suitable test cases to validate grading and topper identification. BTL: BL3 (Apply)

Code:

```java
import java.util.Scanner;

class Student {

    int rollNo;
    String name;
    int m1, m2, m3;
    int total;
    double average;
    char grade;

    Student(int rollNo, String name, int m1, int m2, int m3) {
        this.rollNo = rollNo;
        this.name = name;
        this.m1 = m1;
        this.m2 = m2;
        this.m3 = m3;
    }

    void calculateResult() {
        total = m1 + m2 + m3;
        average = total / 3.0;

        if (average >= 90)
            grade = 'A';
        else if (average >= 75)
            grade = 'B';
        else if (average >= 60)
            grade = 'C';
        else if (average >= 50)
            grade = 'D';
        else
```

```
            grade = 'F';
    }

    void display() {
        System.out.println("--------------------------------");
        System.out.println("Roll No : " + rollNo);
        System.out.println("Name    : " + name);
        System.out.println("Total   : " + total);
        System.out.println("Average : " + average);
        System.out.println("Grade   : " + grade);
    }
}

public class StudentResultAnalyzer {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter Number of Students: ");
        int n = sc.nextInt();

        Student[] students = new Student[n];

        for (int i = 0; i < n; i++) {

            System.out.println("\nEnter Details of Student " + (i + 1));

            System.out.print("Roll No: ");
            int roll = sc.nextInt();
            sc.nextLine();

            System.out.print("Name: ");
            String name = sc.nextLine();

            System.out.print("Mark 1: ");
            int m1 = sc.nextInt();

            System.out.print("Mark 2: ");
            int m2 = sc.nextInt();

            System.out.print("Mark 3: ");
            int m3 = sc.nextInt();

            students[i] = new Student(roll, name, m1, m2, m3);
            students[i].calculateResult();
        }

        System.out.println("\n===== Student Results =====");

        int topper = 0;

        for (int i = 0; i < n; i++) {
            students[i].display();
```

```
            if (students[i].total > students[topper].total)
                topper = i;
        }

        System.out.println("\n===== Class Topper =====");
        System.out.println("Name  : " + students[topper].name);
        System.out.println("Roll No : " + students[topper].rollNo);
        System.out.println("Total : " + students[topper].total);

        sc.close();
    }
}
```

Output:

```
PS C:\Users\WINDOWS 11\OneDrive\Documents\java programs>  & 'C:\Program Files\Java\jdk-17.0.19\bin\ja
va.exe' '-agentlib:jdwp=transport=dt_socket,server=n,suspend=y,address=localhost:57174' '-XX:+ShowCod
eDetailsInExceptionMessages' '-cp' 'C:\Users\WINDOWS 11\AppData\Roaming\Code\User\workspaceStorage\1f
0c1a872a15026a62055ee1093c288c\redhat.java\jdt_ws\java programs_33609ebe\bin' 'StudentResultAnalyzer'

Mark 1: 10
Mark 2: 10
Mark 3: 10

Enter Details of Student 2
Roll No: 12345
Name: yesshu
Mark 1: 9
Mark 2: 9
Mark 3: 9

===== Student Results =====
--------------------------------
Roll No : 123
Name    : yeshanth
Total   : 30
Average : 10.0
Grade   : F
--------------------------------
Roll No : 12345
Name    : yesshu
Total   : 27
Average : 9.0
Grade   : F

===== Class Topper =====
Name  : yeshanth
Roll No : 123
Total : 30
```

10. Smart Home Device Controller using Hierarchical Inheritance

Develop a smart home system with a base class Device and derived classes Light, Fan, and AirConditioner. Implement methods to turn devices on/off and display power consumption. Use polymorphism to control devices through a common reference. Also, construct suitable test cases to validate device operations and power calculations. BTL: BL4 (Analyze)

Code:

```
import java.util.Scanner;

// Base Class
class Device {
```

```
    String name;

    Device(String name) {
        this.name = name;
    }

    void turnOn() {
        System.out.println(name + " is ON");
    }

    void turnOff() {
        System.out.println(name + " is OFF");
    }

    void powerConsumption() {
        System.out.println("Power Consumption");
    }
}

// Light Class
class Light extends Device {

    Light(String name) {
        super(name);
    }

    @Override
    void powerConsumption() {
        System.out.println("Power Consumption : 20 Watts");
    }
}

// Fan Class
class Fan extends Device {

    Fan(String name) {
        super(name);
    }

    @Override
    void powerConsumption() {
        System.out.println("Power Consumption : 75 Watts");
    }
}

// Air Conditioner Class
class AirConditioner extends Device {

    AirConditioner(String name) {
        super(name);
    }

    @Override
    void powerConsumption() {
        System.out.println("Power Consumption : 1500 Watts");
```

```
    }
}

// Main Class
public class SmartHomeController {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        Device device;
        int choice;

        do {

            System.out.println("\n===== Smart Home Device Controller =====");
            System.out.println("1. Light");
            System.out.println("2. Fan");
            System.out.println("3. Air Conditioner");
            System.out.println("4. Exit");
            System.out.print("Enter Choice: ");

            choice = sc.nextInt();
            sc.nextLine();

            switch (choice) {

                case 1:
                    device = new Light("Light");
                    device.turnOn();
                    device.powerConsumption();
                    device.turnOff();
                    break;

                case 2:
                    device = new Fan("Fan");
                    device.turnOn();
                    device.powerConsumption();
                    device.turnOff();
                    break;

                case 3:
                    device = new AirConditioner("Air Conditioner");
                    device.turnOn();
                    device.powerConsumption();
                    device.turnOff();
                    break;

                case 4:
                    System.out.println("Thank You!");
                    break;

                default:
                    System.out.println("Invalid Choice.");
            }
```

```
        } while (choice != 4);

        sc.close();
    }
}
```

Output:

```
PS C:\Users\WINDOWS 11\OneDrive\Documents\java programs>  & 'C:\Program Files\Java\jdk-17.0.19\bin\java.exe' '-agentlib:jdwp=transport=dt_socket,server=n,suspend=y,address=localhost:58969' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\WINDOWS 11\AppData\Roaming\Code\User\workspaceStorage\1f0c1a872a15026a62055ee1093c288c\redhat.java\jdt_ws\java programs_33609ebe\bin' 'SmartHomeController'

===== Smart Home Device Controller =====
1. Light
2. Fan
3. Air Conditioner
4. Exit
Enter Choice: 1
Light is ON
Power Consumption : 20 Watts
Light is OFF

===== Smart Home Device Controller =====
1. Light
2. Fan
3. Air Conditioner
4. Exit
Enter Choice: 2
Fan is ON
Power Consumption : 75 Watts
Fan is OFF

===== Smart Home Device Controller =====
1. Light
2. Fan
3. Air Conditioner
4. Exit
Enter Choice: 3
Air Conditioner is ON
Power Consumption : 1500 Watts
Air Conditioner is OFF
```